

shogiAI 專案報告

將棋 CSA 棋譜資料庫與數據科學模型設計

專案名稱：shogiAI

2026 年 5 月

摘要

本專案以日本將棋 CSA 棋譜為核心資料，建立一套從棋譜蒐集、格式解析、資料庫正規化、局面重建、訓練資料匯出、policy/value 神經網路訓練，到網頁棋譜瀏覽與 AI 對弈的完整流程。系統目前包含 1000 份 CSA 棋譜，並由資料庫匯出 111238 筆局面樣本，每筆樣本包含 $43 \times 9 \times 9$ 的局面特徵、13689 類走法標籤與勝負價值標籤。本報告分成三個層次：第一部分說明整體主系統與製作流程；第二部分以資料庫課程角度說明關聯式設計、正規化、匯入與查詢；第三部分以數據科學課程角度說明特徵工程、監督式學習、CNN policy/value model，以及與 alpha-beta 搜尋整合的理論方法。

目录

1	主要系統設計	2
1.1	專案目標	2
1.2	專案檔案與模組分工	2
1.3	整體資料流程	3
1.4	系統架構	3
1.5	製作流程回顧	3
2	資料庫課程報告：CSA 棋譜資料庫設計	5
2.1	資料庫設計目標	5
2.2	關聯式資料模型	5
2.3	ER 關係說明	5
2.4	正規化設計	6
2.5	資料表欄位設計	6
2.5.1	棋局主檔：game_records	6
2.5.2	走法表：moves	7

目錄	2
2.5.3 局面表：positions	7
2.6 匯入流程與交易控制	7
2.7 查詢與 API 設計	8
2.8 資料庫設計評估	8
3 數據科學課程報告：將棋 Policy/Value 模型	9
3.1 數據科學問題定義	9
3.2 資料來源與資料規模	9
3.3 局面特徵工程	10
3.4 走法標籤設計	10
3.5 Value 標籤設計	10
3.6 模型架構	11
3.7 損失函數與訓練方法	11
3.8 目前訓練結果	12
3.9 傳統搜尋與神經網路整合	12
3.9.1 局面評估	12
3.9.2 Negamax Alpha-Beta	13
3.9.3 搜尋優化	13
3.10 數據科學流程評估	13
4 兩門課程的分工總結	14
4.1 資料庫課程重點	14
4.2 數據科學課程重點	14
4.3 共同成果	14
5 結論	14
A 附錄：主要執行指令	15
A.1 匯入 CSA 至 MySQL	15
A.2 由 MySQL 匯出訓練資料	15
A.3 訓練 Policy/Value Network	15
A.4 啟動棋譜瀏覽與 AI 對弈 API	15

1 主要系統設計

1.1 專案目標

shogiAI 的核心目標不是只製作一個能讀取棋譜的工具，而是建立一條可重複執行的資料管線，使原始 CSA 棋譜能同時服務於資料庫管理、棋譜瀏覽、數據分析與 AI 模型訓練。整體設計可分為下列目標：

- 將原始 CSA 棋譜轉換為乾淨、可查詢、可重建局面的資料。
- 以 MySQL 保存棋局、棋手、每手走法與每個局面，使棋譜不只是檔案，而是可被條件查詢的結構化資料。
- 由資料庫重建訓練資料，產生 policy/value network 可使用的 NumPy 壓縮資料集。
- 使用 CNN 學習棋手在特定局面下的走法選擇與勝負估計。
- 在網頁介面中提供棋譜瀏覽、自行對弈、AI 對弈、policy 候選手與搜尋資訊。

1.2 專案檔案與模組分工

檔案或目錄	功能
data/*.csa	原始 CSA 棋譜資料，共 1000 份。
src/csa_preprocess.py	解析 CSA/KIF、驗證合法走法、編碼局面與走法、輸出 .npz 訓練樣本。
src/import_csa_to_db.py	將 CSA 棋譜匯入 MySQL，建立 users、players、game_records、moves、positions 五張表。
src/export_policy_data.py	從 MySQL 讀取局面與走法，重建 policy/value 訓練資料。
src/policy_model.py	定義 CNN policy/value network 與推論封裝。
src/train_policy.py	執行資料切分、模型訓練、驗證、測試與 checkpoint 儲存。
src/ai_search.py	實作將棋局面評估、negamax alpha-beta 搜尋、quiescence search、transposition table、killer/history move ordering。
src/csa_browser_api.py	提供 HTTP API，支援從 MySQL 或本機 CSA 檔讀取棋局，並連接 AI 搜尋與 policy model。
web/index.html, web/app.js, web/styles.css	前端介面，包含棋譜瀏覽、自行對弈、AI 對弈與資料庫統計。

檔案或目錄	功能
out/policy_dataset.npz	從資料庫匯出的訓練資料，共 111238 筆樣本。
out/policy_model.pt	已訓練完成的 PyTorch 模型 checkpoint。

1.3 整體資料流程

整體流程可視為一條 ETL 與機器學習管線：

1. **Extract**：從 data 目錄讀取 CSA 棋譜。
2. **Transform**：使用 cshogi 解析棋譜，檢查每手是否合法，並在每手之前保存局面。
3. **Load**：將棋局、棋手、走法、局面寫入 MySQL 正規化資料表。
4. **Dataset Export**：從資料庫重新組合局面與下一手，轉換成 CNN 可訓練的張量資料。
5. **Model Training**：訓練 policy/value network，學習走法分類與局面勝率估計。
6. **Serving**：透過 csa_browser_api.py 提供棋譜瀏覽、AI 對弈與模型候選手分析。

1.4 系統架構

系統架構的設計重點是分離「資料保存」、「模型訓練」與「使用者互動」。CSA 原始檔只作為初始資料來源；MySQL 成為查詢、重建與後續分析的主要資料層；模型與搜尋則透過 API 被前端呼叫。

CSA Files → Parser / Validator → MySQL → NPZ Dataset → CNN Model → Search + Web API → I

此架構的優點是每一層都可以獨立測試。若棋譜匯入失敗，可檢查資料庫層；若模型效果不佳，可重新匯出資料或調整訓練；若前端呈現錯誤，可直接呼叫 API 驗證 JSON 回應。

1.5 製作流程回顧

本專案的製作流程大致經過下列階段：

1. 建立 CSA 棋譜前處理程式，先確認可以把棋譜轉成合法局面與下一手。
2. 擴充 MySQL 匯入流程，將原本檔案式資料改成資料表設計。
3. 建立棋譜瀏覽 API 與網頁前端，用資料庫資料還原棋盤、持駒、手數與走法列表。

4. 由資料庫匯出 policy/value 訓練資料，讓資料科學流程不直接依賴原始檔案。
5. 建立 CNN 模型，加入 value mask、AdamW、cosine learning rate scheduler、依棋局切分資料集等訓練設計。
6. 建立 AI 搜尋，將傳統 alpha-beta 搜尋與神經網路 policy ordering、value head 評估結合。

2 資料庫課程報告：CSA 棋譜資料庫設計

2.1 資料庫設計目標

資料庫部分的核心問題是：CSA 棋譜原本是半結構化文字檔，雖然適合保存對局紀錄，但不適合查詢、統計與局面重建。因此資料庫設計需要達成下列目標：

- 保存棋局層級資訊，例如賽事、場地、日期、開局、勝負結果與原始檔名。
- 保存棋手資訊，避免同一棋手在多個棋局中重複出現文字資料。
- 保存每一手走法，使系統能依手數回放棋局。
- 保存每一手之後的 SFEN 局面，使前端與資料科學流程可以快速取得任意局面。
- 支援條件查詢，例如依賽事、日期區間、棋手、開局搜尋棋局。
- 支援去重匯入，避免同一份 CSA 被重複寫入。

2.2 關聯式資料模型

本專案資料庫共使用五張主要資料表：

資料表	設計目的
users	保存上傳者或匯入者資訊。雖然目前主要由系統匯入，但保留使用者表可支援未來多人上傳。
players	保存棋手姓名與段位。棋手被抽離成獨立資料表，避免在每筆棋局中重複保存字串。
game_records	保存棋局主檔，包含黑方、白方、賽事、場地、開局、勝負、原始檔名與日期。
moves	保存每一手走法，包含手數、輪到哪一方、CSA/USI 表示、用時與註解。
positions	保存每一手後的局面，包含手數、輪到哪一方、SFEN、hash、是否被將軍與合法手數量。

2.3 ER 關係說明

主要關聯如下：

- users.user_id 一對多連到 game_records.uploader_id。
- players.player_id 分別連到 game_records.black_player_id 與 game_records.white_player_id。

- `game_records.game_id` 一對多連到 `moves.game_id`。
- `game_records.game_id` 一對多連到 `positions.game_id`。

以概念式 ER 模型表示：

$$\text{User} \xrightarrow{1:N} \text{GameRecord} \xleftarrow{N:1} \text{Player}$$

$$\text{GameRecord} \xrightarrow{1:N} \text{Move}, \quad \text{GameRecord} \xrightarrow{1:N} \text{Position}$$

2.4 正規化設計

本資料庫設計主要符合第三正規化 (Third Normal Form, 3NF) 的精神：

- 第一正規化：每個欄位保存單一語意，例如棋手姓名、日期、走法、SFEN 分開保存。
- 第二正規化：每張表的非鍵欄位都依賴該表主鍵，例如 `moves` 的 `usi_move` 依賴 `move_id`。
- 第三正規化：棋手姓名不直接重複放在 `game_records` 中，而是透過 `players` 外鍵參照，避免傳遞相依。

這樣設計的好處是資料一致性較高。例如同一棋手出現在多場比賽時，資料庫只需保存一份棋手資料。若未來要統計某位棋手的勝負或常用開局，也能直接由 `players` 與 `game_records` 關聯查詢。

2.5 資料表欄位設計

2.5.1 棋局主檔：game_records

`game_records` 是整個資料庫的中心表。其欄位包含：

- `game_id`：主鍵。
- `black_player_id`, `white_player_id`：黑方與白方棋手外鍵。
- `event_name`, `site`, `opening`：CSA metadata。
- `result`：勝負結果，例如 `TORYO`, `DRAW`, `SENNICHITE`。
- `source_format`：資料來源格式，目前為 `CSA`。
- `original_file_name`：原始檔名，用於追蹤來源與去重。
- `played_at`：對局日期。

2.5.2 走法表：moves

`moves` 保存棋局的時間序列。每一列是一手棋：

- `move_number`：第幾手。
- `side_to_move`：此手由黑方或白方下。
- `original_move` 與 `usi_move`：保存原始走法與標準化 USI 表示。
- `move_time`, `comment`：保存棋譜附加資訊。

2.5.3 局面表：positions

`positions` 是本專案資料庫設計中最重要的表之一。傳統棋譜只保存走法，但本專案額外保存每一手後的 SFEN 局面，使系統可以快速取得第 n 手局面，而不必每次從第一手重播。

- `move_number`：此局面對應到第幾手後；0 代表初始局面。
- `sfen`：完整局面表示，可被 `cschogi.Board` 直接重建。
- `sfen_hash`：局面 hash，可用於重複局面偵測與索引。
- `is_check`：是否正在被將軍。
- `legal_moves_count`：該局面合法手數量，可用於分析複雜度。

2.6 匯入流程與交易控制

`import_csa_to_db.py` 的匯入流程如下：

1. 掃描輸入路徑中的 CSA 檔案。
2. 使用 `cschogi.CSA.Parser` 解析棋譜。
3. 讀取棋手、賽事、日期與結果等 metadata。
4. 透過 `ensure_player` 避免重複建立棋手。
5. 寫入 `game_records` 後取得 `game_id`。
6. 從初始局面開始逐手檢查合法性，建立 `moves` 與 `positions`。
7. 使用交易 `commit` 確保每局資料完整寫入；若發生錯誤則 `rollback`。

此設計使用 `--skip-existing` 依 `original_file_name` 跳過已匯入棋局，並提供 `--max-games` 與 `--pause-every` 來因應學校 MySQL 可能存在的查詢限制。

2.7 查詢與 API 設計

前端使用 `csa_browser_api.py` 提供的 API 存取資料。主要端點包括：

API	用途
GET <code>/api/games</code>	取得棋局清單，可依賽事、日期、棋手、開局篩選。
GET <code>/api/games/<id>?ply=N</code>	取得指定棋局第 N 手後的局面。
GET <code>/api/db/stats</code>	回傳資料庫棋局數、棋手數、走法數、局面數與重複檔案統計。
POST <code>/api/self-play/state</code>	由前端傳入 USI 走法序列，後端重建自行對弈局面。
POST <code>/api/ai-play/move</code>	呼叫 AI 搜尋取得下一手。
POST <code>/api/policy/candidates</code>	回傳 policy model 對目前合法手的候選排序。

在 SQL 查詢上，棋局清單使用 `LEFT JOIN players` 取得黑白棋手，並使用 `COUNT(m.move_id)` 統計手數。篩選條件採用參數化查詢，例如棋手搜尋會對黑方與白方姓名做 `LIKE` 比對，避免直接拼接使用者輸入造成 SQL injection。

2.8 資料庫設計評估

本資料庫設計的優點如下：

- **可重建性**：保存每一手局面，使任意手數可直接回放。
- **可查詢性**：棋局、棋手、走法、局面分表後，可針對不同層級做查詢。
- **資料一致性**：使用外鍵維持棋局與棋手、走法、局面的關係。
- **可擴充性**：未來可新增 `openings`、`tournaments`、`model_predictions` 等表。
- **可服務數據科學**：`positions` 與 `moves` 可直接 JOIN 成監督式學習資料。

後續若要優化資料庫效能，建議加入下列索引：

```
CREATE INDEX idx_game_records_played_at ON game_records(played_at);
CREATE INDEX idx_game_records_original_file ON game_records(original_file_name);
CREATE INDEX idx_moves_game_number ON moves(game_id, move_number);
CREATE INDEX idx_positions_game_number ON positions(game_id, move_number);
CREATE INDEX idx_positions_sfen_hash ON positions(sfen_hash);
```

3 數據科學課程報告：將棋 Policy/Value 模型

3.1 數據科學問題定義

數據科學部分的任務可視為監督式學習問題。給定一個將棋局面 s ，模型需要學習兩個目標：

1. **Policy prediction**：預測棋手在局面 s 下會選擇哪一手 a 。
2. **Value prediction**：預測目前輪到下棋的一方最後是否會獲勝。

因此每筆資料可表示為：

$$(s_i, a_i, z_i)$$

其中 s_i 是局面張量， a_i 是實戰下一手的分類標籤， $z_i \in \{-1, 0, 1\}$ 是從目前下棋方視角定義的勝負結果。

3.2 資料來源與資料規模

目前專案使用 1000 份 CSA 棋譜，經資料庫匯出後形成：

項目	數值
原始 CSA 棋譜數	1000 份
訓練樣本數	111238 筆
局面張量形狀	(111238, 43, 9, 9)
走法標籤數	13689 類
勝負標籤樣本數	111238 筆
訓練集	88473 筆
驗證集	11166 筆
測試集	11599 筆

資料集中結果分布如下：

結果類型	樣本數
TORYO	110967
DRAW	104
SENNICHITE	167

3.3 局面特徵工程

將棋棋盤為 9×9 ，但單純用一個整數矩陣不足以表達棋子種類、敵我關係、持駒與輪到誰下。因此本專案採用多平面 one-hot / count encoding，將每個局面編成 $43 \times 9 \times 9$ 張量。

- 前 28 個平面：棋盤上雙方棋子種類。14 種棋子類型乘上己方與對方。
- 第 29 到 35 個平面：己方持駒數量，共 7 類可打入棋子。
- 第 36 到 42 個平面：對方持駒數量。
- 第 43 個平面：輪到哪一方下，或在旋轉視角下作為 side-to-move 輔助資訊。

設計上預設使用 **side-to-move orientation**。也就是當白方下棋時，局面與走法會旋轉 180 度，使模型永遠從「目前下棋方」角度看棋盤。這樣可以減少黑白對稱造成的資料稀疏，讓同一種戰術型態不會因方向不同而被模型視為完全不同的模式。

3.4 走法標籤設計

Policy head 需要將合法走法轉成固定類別。將棋走法分為一般移動與打入持駒：

- 一般移動：來源格 81 種、目標格 81 種、是否升變 2 種，共 $81 \times 81 \times 2 = 13122$ 類。
- 打入持駒：7 種可打入棋子、目標格 81 種，共 $7 \times 81 = 567$ 類。

總標籤數為：

$$81 \times 81 \times 2 + 7 \times 81 = 13689$$

雖然在任一局面中合法手遠少於 13689 類，但固定輸出空間使神經網路可用標準 cross entropy loss 訓練。

3.5 Value 標籤設計

Value label 從目前下棋方視角定義：

$$z = \begin{cases} 1, & \text{目前下棋方最後獲勝} \\ 0, & \text{和棋} \\ -1, & \text{目前下棋方最後落敗} \end{cases}$$

CSA 中的 TORYO 表示投了，實作上會根據最後局面的 side_to_move 推定勝方；DRAW 與 SENNICHITE 則標記為 0。後來加入的 value_masks 用來表示哪些樣本有可靠 value 標籤，避免未來遇到未知結果時強迫模型學習錯誤標籤。目前資料集中 111238 筆樣本皆有 value 標籤。

3.6 模型架構

本專案使用小型 CNN policy/value network。輸入為 $43 \times 9 \times 9$ 張量，先經過三層 convolution 抽取局部棋型特徵，再分成 policy head 與 value head。

$$\begin{aligned} x &\in \mathbb{R}^{43 \times 9 \times 9} \\ h &= f_{\text{conv}}(x) \\ \hat{p} &= f_{\text{policy}}(h) \in \mathbb{R}^{13689} \\ \hat{v} &= f_{\text{value}}(h) \in [-1, 1] \end{aligned}$$

模型結構如下：

- Feature extractor：Conv2d(43,64)、ReLU、Conv2d(64,64)、ReLU、Conv2d(64,32)、ReLU。
- Policy head：flatten 後接 Linear(32*9*9,512)、ReLU、Linear(512,13689)。
- Value head：flatten 後接 Linear(32*9*9,128)、ReLU、Linear(128,1)、Tanh。

CNN 適合本任務的原因是棋盤具有明顯空間結構。棋子的攻防、升變區、王周圍安全性與棋子效率都與局部相對位置有關，而 convolution 能用共享權重學習這些局部模式。

3.7 損失函數與訓練方法

訓練時同時最佳化 policy loss 與 value loss：

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + \lambda \mathcal{L}_{\text{value}}$$

其中：

$$\mathcal{L}_{\text{policy}} = -\log \frac{\exp(\hat{p}_a)}{\sum_j \exp(\hat{p}_j)}$$

$$\mathcal{L}_{\text{value}} = m \cdot (\hat{v} - z)^2$$

a 是實戰走法標籤， z 是勝負標籤， m 是 value mask， λ 為 value-loss-weight，目前設定為 0.25。

訓練策略包含：

- 使用 AdamW optimizer，降低 overfitting 並加入 weight decay。
- 使用 cosine annealing learning rate scheduler，使學習率逐步下降。

- 以棋局為單位切分 train/validation/test，避免同一盤棋的相鄰局面同時出現在訓練與測試中造成資料洩漏。
- 使用 Top-1、Top-3、Top-5 accuracy 評估 policy 預測，使用 MSE 與 MAE 評估 value 預測。

3.8 目前訓練結果

目前模型訓練 12 epochs，checkpoint 中保存的測試集結果如下：

指標	數值
Test policy loss	4.4340
Top-1 accuracy	0.2230
Top-3 accuracy	0.3695
Top-5 accuracy	0.4385
Value loss	1.0876
Value MAE	0.9263

Top-1 accuracy 約 22.3% 代表模型直接猜中實戰下一手的比例。由於將棋合法手分支大、同一局面可能存在多個合理手，因此 Top-3 與 Top-5 更能反映 policy 是否把合理候選手排在前面。目前 Top-5 約 43.8%，可作為 AI 搜尋的 move ordering 輔助。

3.9 傳統搜尋與神經網路整合

本專案不是只使用神經網路直接下棋，而是將 neural policy/value 與傳統 alpha-beta 搜尋結合。

3.9.1 局面評估

ai_search.py 中設計了手工評估函數，包含：

- 棋子價值：步、香、桂、銀、金、角、飛、王與升變棋子。
- 持駒價值：將棋特有的打入資源。
- 棋子位置價值：升變區、中央控制、棋子前進程度。
- 王安全：王周圍守備、敵方攻擊、線性壓力。
- 機動性：可走步數與大子活動能力。
- 懸空棋與棋子效率：被攻擊但缺乏保護的棋子會扣分。

3.9.2 Negamax Alpha-Beta

搜尋採用 negamax 表示法，因為雙人零和遊戲中，對手的最佳分數可視為自己的負分：

$$V(s) = \max_{a \in A(s)} -V(T(s, a))$$

alpha-beta pruning 使用上下界 α 與 β 剪去不可能影響最終選擇的分支。此方法能在相同時間內搜尋更深層局面。

3.9.3 搜尋優化

專案中實作了多種搜尋優化：

- **Transposition table**：使用 zobrist hash 保存已搜尋局面，避免重複計算。
- **Quiescence search**：在深度到達時繼續搜尋吃子或升變等戰術手，降低 horizon effect。
- **Killer move heuristic**：保存曾造成 beta cutoff 的安靜手，提升同層搜尋排序。
- **History heuristic**：對過去有效的走法累積分數，用於排序。
- **Policy move ordering**：使用神經網路 policy 將合法手排序，讓 alpha-beta 優先搜尋模型認為較合理的手。
- **Value head bonus**：可在根節點加入 value head 評估，補強手工評估函數。

3.10 數據科學流程評估

本專案的數據科學設計有三個重要特點：

1. **資料不是直接從檔案訓練，而是從資料庫匯出**。這讓資料清洗、查詢、過濾與統計更容易追蹤。
2. **模型同時學 policy 與 value**。Policy 用於模仿棋手下一手，value 用於估計局面好壞，兩者可同時服務搜尋。
3. **資料切分以棋局為單位**。避免相鄰局面洩漏到測試集，使評估更接近真實泛化能力。

目前限制則包括：

- 資料量仍相對有限，1000 盤棋對深度學習將棋模型而言只適合作為小型原型。

- Value label 只由終局勝負回推，沒有使用中盤局面的人類評估或引擎分數，因此 value head 學習較困難。
- 目前 CNN 架構較小，尚未使用 residual blocks、attention 或合法手 mask loss。
- Policy loss 仍對所有 13689 類輸出做分類，推論時才在合法手內排序；未來可加入合法手遮罩改善學習效率。

4 兩門課程的分工總結

4.1 資料庫課程重點

資料庫課程的貢獻在於把 CSA 棋譜從文字檔轉換為關聯式資料庫。重點包含 ER model、正規化、外鍵關聯、交易控制、參數化查詢、資料去重與 API 查詢。此部分強調「如何設計資料結構，使棋譜可以被可靠保存與查詢」。

4.2 數據科學課程重點

數據科學課程的貢獻在於把資料庫中的棋譜轉換為機器學習樣本。重點包含特徵工程、走法標籤設計、勝負標籤、CNN policy/value network、訓練/驗證/測試切分、評估指標，以及與 alpha-beta 搜尋的整合。此部分強調「如何從結構化資料中學習決策模型」。

4.3 共同成果

兩門課雖然重點不同，但在本專案中互相依賴。資料庫提供乾淨、可追蹤、可查詢的資料來源；數據科學模型則將資料轉化為可用於 AI 對弈的決策能力。最終網頁系統把兩者整合成可互動展示的應用：使用者可以查棋譜、看局面、搜尋棋局、自己下棋，也可以與 AI 對弈並觀察模型候選手。

5 結論

shogiAI 專案展示了一個從資料工程到人工智慧應用的完整流程。資料庫部分解決了棋譜保存、查詢與重建問題；數據科學部分則將這些資料轉換成神經網路可學習的 policy/value 任務；主系統再透過 API 與前端把資料與模型串接成可操作的產品。此專案的價值不只在於完成一個將棋 AI，而是在於建立一套可擴充的資料與模型基礎。未來若加入更多棋譜、更強的模型架構、合法手遮罩、引擎評分資料與更完整的資料庫索引，系統可以進一步提升棋力、查詢效能與分析能力。

A 附錄：主要執行指令

A.1 匯入 CSA 至 MySQL

```
python src\import_csa_to_db.py ^  
  --input data ^  
  --recursive ^  
  --create-tables ^  
  --host 140.135.65.53 ^  
  --port 3306 ^  
  --user 11211213 ^  
  --database DB11211213 ^  
  --skip-existing
```

A.2 由 MySQL 匯出訓練資料

```
python src\export_policy_dataset.py ^  
  --output out\policy_dataset.npz ^  
  --host 140.135.65.53 ^  
  --port 3306 ^  
  --user 11211213 ^  
  --password <password> ^  
  --database DB11211213
```

A.3 訓練 Policy/Value Network

```
python src\train_policy.py ^  
  --input out\policy_dataset.npz ^  
  --output out\policy_model.pt ^  
  --batch-size 256 ^  
  --value-loss-weight 0.25
```

A.4 啟動棋譜瀏覽與 AI 對弈 API

```
python src\csa_browser_api.py ^  
  --host 127.0.0.1 ^  
  --port 8000 ^  
  --source mysql ^
```

```
--db-host 140.135.65.53 ^  
--db-port 3306 ^  
--db-user 11211213 ^  
--db-name DB11211213 ^  
--policy-model out\policy_model.pt ^  
--value-weight 200 ^  
--policy-order-ply 1
```